MathWorks EDG Intern — Manager Interview Prep

Mahip Parekh | Complete Guide

---

1. INTRODUCTION (~90 seconds)

Hi, I'm Mahip. I'm a Master's student in Computer Science at Northeastern University in Boston. I'm currently in my third semester.

Before coming to the US, I worked as a software engineer in India for about a year. I was on a product support team, where we spent a lot of time resolving issues, maintaining the product, and supporting upgrades. That experience taught me how important clear communication is, especially when you're helping other people troubleshoot and get unblocked.

Since starting grad school, I've built projects across a wide range. I built a full stack learning management system. I also built a distributed file storage system in C++ focused on fault tolerance. And I built a Java based calendar management system, where I focused on clean MVC design and rigorous testing.

This semester I'm also learning AWS and machine learning. I'm excited to build something meaningful at the intersection of both.

Outside of tech, I'm into coffee and cooking. I grind my own beans because I love the fresh smell of coffee. And I recently realized I can cook almost as good as my mom.

What drew me to the EDG program is the mix of strong engineering culture, collaboration, and continuous learning. I know as an intern I'll be focused on one team, but I really like that EDG supports long term growth over time. That's the kind of environment where I know I'll learn fast and contribute strongly.

---

2. WHY MATHWORKS / WHY EDG

MathWorks is the company that set the standard for simulation software — MATLAB and Simulink are foundational tools for engineers and scientists worldwide. What excites me is that any work I do here, even as an intern, eventually touches someone who's using these tools to push their field forward. I may not be the one doing the research, but I'm helping the person who is — and that kind of indirect impact means a lot to me.

As for EDG specifically, I like that it's not just a "here's your desk, go code" kind of program. You're set up to learn deeply — whether that's through supporting real users and strengthening your problem-solving by helping someone who's stuck, or working with a team on improving the product itself. Either way, you're learning by doing meaningful work, not busywork. That's the kind of environment I thrive in.

---

3. STAR: INITIATIVE / AUTOMATION (IDX)

Use for: "Time you took initiative," "Something you're proud of," "Leadership"

Situation: Our team was asked to produce a report showing which version of Drupal and our company's product each client website was running. We had hundreds of websites across dev, staging, and production — over 900 total. The only way to check was to manually log into each website's backend, navigate to the status page, and note down the versions. At that scale, doing it by hand would have taken ages and been error-prone.

Action: I proposed automating the whole thing and took ownership of it. I wrote a script in Java using Selenium that would go through our internal list of client websites, log into each one, pull the version info from the backend, and output everything into a clean Excel sheet. I built three versions for dev, staging, and production environments.

Halfway through, I ran into something unexpected — a bunch of old dev and staging URLs were completely dead. Either the client had left or their environments had been deleted, but there was no single place that tracked this. I flagged this to my manager, and he realized we could hit two birds with one stone — not only would we get the version report, but we'd also finally have a clean, accurate list of which client websites were actually still active so we could clear out the old ones from our servers.

Result: I got version details for every active client website — 400+ dev, 300+ staging, 270+ production. The dead URLs were identified and flagged for cleanup. My manager and team lead asked me to document the script on Confluence and push it to Bitbucket so others could reuse it. We also discussed how to improve it so it wouldn't need a laptop running continuously. And the report ended up being useful beyond just our team — other teams asked us to share it for their own purposes.

Follow-ups:

- **"Why Java/Selenium?"** → I was comfortable with Java, and Selenium was the natural choice for browser-based automation.
- **"What was the hardest part?"** → Handling dead URLs and edge cases across 900+ sites.
- **"What would you do differently?"** → The script had to run on my laptop for about a week. If doing it again, I'd batch the Excel data in memory and write in bulk, and move it to a server with a scheduled job. My team lead and I actually discussed these improvements — I optimized the Excel writing to bulk insertion, and my team lead handled the hosting side since he had more experience with our infra. The final version didn't need someone babysitting a laptop anymore.

---

4. STAR: TEAM CONFLICT (Calendar Project)

Use for: "Conflict with a teammate," "Disagreement and how you resolved it," "Differences of opinion"

Situation: For the final phase of our semester-long calendar management project, we had to add several new features and build an entirely new GUI desktop interface — up until that point it had been a command-line application. It was a big scope increase, and our goal was 100% mutation test coverage.

Task: My teammate and I disagreed on how to approach testing. He wanted to implement all the features first, then dedicate the remaining time to writing tests. I was concerned about leaving all the testing to the end because our target was 100% mutation coverage — that's a lot of test writing to cram into whatever time is left.

Action: We went back and forth for a while, and then I took a step back and said — let's figure out what's actually important here. Turns out both were important — we needed to move fast on implementation *and* we couldn't risk falling behind on testing. So I proposed a middle ground: we'd test each feature's core working behavior as we implemented it, but save edge case testing for later. That way we'd never have a pile of untested code, but we also wouldn't slow down implementation by chasing every edge case upfront. We split the workload equally and worked in parallel.

Result: We finished before the deadline with 100% functionality and 98% mutation test coverage. The approach worked because neither of us "won" — we found something that addressed both concerns.

Follow-ups:

- **"Why were you concerned about testing at the end?"** → With mutation testing at 100% target, you need tests strong enough to catch mutants. That takes time and thought. Cramming it would mean weaker tests.
- **"How did you split the work?"** → We made a checklist of features. I'd take two or three and implement them while my teammate wrote tests for the ones I'd just finished. Then we'd swap. At the end, we paired up to write remaining edge case tests and fix bugs.
- **"What did you learn?"** → When you disagree, stop debating the approach and align on what matters most first. Once we agreed that both speed and test quality were non-negotiable, the compromise came naturally. Rotating responsibilities keeps things fair and keeps both people sharp.
- **"Would you do anything differently?"** → I would have suggested the priority-based approach sooner instead of going back and forth.

---

5. STAR: FAILURE / MISTAKE (IDX — Client Escalation)

Use for: "Time you failed," "A mistake and what you learned," "Received negative feedback"

Situation: At IDX, after every client website upgrade, we had a checklist of quality checks to verify the upgraded site was working properly. It was my responsibility to go through each checkpoint, and only when I gave the green flag would the hosting team map the live URL to the newly upgraded production site. During one of these post-upgrade checks, I was assigned a UI bug — the website administrator couldn't publish an uploaded image. It should have been as simple as uploading and clicking "publish," but it wasn't working.

Task: Before testing, I actually disabled the client email notification toggle in the backend — so I thought I was safe to click around. But because the site's cache was stale, my toggle change never took effect. So when I clicked the publish button multiple times to debug, each click fired off an email to the actual client saying an image was published and a page was updated. The client started receiving these unexpected emails and raised a concern — they hadn't asked anyone to make changes to their live site. It got escalated, and a meeting was called between the client's manager and our team.

Action: Before that meeting, I sat down with my team and walked them through exactly what happened and what I thought was causing the issue. We figured out that the client website's cache on our server needed to be cleared — that's what was causing the weird behavior. In the meeting, I explained the situation honestly and gave a live demonstration showing how a website behaves when its server-side cache isn't cleared — how it doesn't reflect updates properly and acts unpredictably. My manager and team lead backed me up, and together we addressed the client's concerns.

Result: The issue was resolved, the client was notified with a clear explanation, and things moved on smoothly. I learned that taking a precaution isn't enough — you need to verify it actually worked. I assumed the email toggle was off because I turned it off, but the cache meant it never took effect. Since then, I always verify my changes are reflected before proceeding, especially on live environments.

Follow-ups:

- **"How did you feel when you found out?"** → Honestly, a bit panicked. But I knew the worst thing I could do was hide it. I went to my team immediately.
- **"Did your manager react negatively?"** → No, he was supportive. I think because I came to him with the problem *and* we already had the root cause figured out before the meeting.
- **"What did you change after this?"** → I stopped assuming my changes took effect just because I made them. Especially on live sites with caching, I now always verify the change is actually reflected before moving forward.
- **"Why did you demo the cache issue?"** → I wanted the client's manager to *see* the issue, not just hear me explain it. Showing it live made it easier for everyone to understand.

---

6. STAR: WEAKNESS / RECEIVING FEEDBACK (IDX — Asking for Help)

Use for: "What's your weakness?", "Negative feedback," "How do you develop yourself?"

Situation: This was my first professional job, right out of college. I joined the product-support team at IDX and was given access to our company's product codebase to ramp up and understand how the product worked. I had this mindset from college — I can figure everything out myself, I don't need to ask for help.

Task: So I spent two weeks going through the codebase on my own. If I didn't understand something, I'd Google it. I was sitting right next to my team in an open workspace, but I never once reached out to ask anyone a question. After two weeks, I reported to my manager and explained the product as best as I understood it.

Action: He was disappointed — he told me I was too slow and hadn't learned enough. And he knew exactly why. He addressed it directly but in a really supportive way — essentially saying, we're a team and we've got your back. That clicked for me. I changed my approach completely. I started reaching out to my senior manager — I'd message him to check if he was free, walk over to his desk, sit beside him and have a conversation about how the product worked. Those conversations didn't just answer my existing questions — they surfaced things I hadn't even discovered yet on my own.

Result: My ramp-up accelerated dramatically after that. I went from slowly scrolling through code in isolation to actually understanding the product through the people who built it. It taught me that asking for help isn't a weakness — it's how you learn faster and build relationships with your team at the same time.

Follow-ups:

- **"Do you still struggle with this?"** → My instinct to try things on my own is still there, but now I set a mental timer. If I'm stuck for more than 20-30 minutes, I reach out. The instinct to be independent isn't bad — I just needed to learn when to switch from solo mode to team mode.
- **"Why did you have that mindset?"** → In college, most work is individual. You're graded on what *you* know. It took a professional environment to teach me that the goal is the team's output, not proving I can do it alone.
- **"How did you feel when your manager said that?"** → A bit embarrassed. But he said it in a way that wasn't harsh — it was clear he wanted to help me, not criticize me.

---

7. STAR: LEARNED A NEW SKILL (Distributed File System)

Use for: "Time you learned a new skill," "Outside your comfort zone," "Challenging project"

Situation: For my Building Scalable Distributed Systems course, my partner and I had to build a distributed file storage system from scratch in C++. I knew concepts like consistent hashing, chain replication, and thread pools theoretically from class — but I had never implemented any of them. And I had zero experience with TCP socket programming.

Task: We had to build a system with three layers — a client that splits files into chunks, a metadata layer using chain replication for consistency, and a storage layer using a consistent hash ring for data distribution. It needed to handle concurrent clients, survive node failures, and verify file integrity.

Action: We split the work so I focused on building the networking foundation while my partner planned the next steps — what to implement next and how things would fit together architecturally. As I started writing the

raw TCP socket code, I quickly realized it was getting messy, so I refactored everything into wrapper classes — a clean abstraction layer around connect, send, receive, and all the byte-ordering details so the rest of our code didn't have to deal with low-level socket calls directly.

The hardest bug I ran into was that messages sent from server to client weren't arriving the way they should — I couldn't deserialize them properly on the receiving end. I eventually realized the problem was that there was no way to know where one message ended and the next began. So I came up with a format — prepend each message with its length, so the receiver always knows how many bytes to read before deserializing. Once that clicked, the rest of the communication layer fell into place.

Result: The final system handled 128+ concurrent client connections, achieved sub-200ms latency for file operations, 99.5% availability during node failures, and a 3.2x throughput improvement. We ran automated fault tolerance tests where we'd kill a storage node mid-operation and verify the file could still be recovered from a replica.

Follow-ups:

- **"How did you split the work?"** → I built the networking foundation — TCP socket wrapper classes, serialization/deserialization format. My partner focused on architectural planning. From there we both worked across the layers.
- **"What was the hardest part?"** → Message deserialization — solved by length-prefixed message format.
- **"How did you go from theory to implementation?"** → By building and hitting walls. I knew TCP sockets conceptually, but once I started writing the raw code, it got messy fast. That's when I realized I needed wrapper classes. The learning came from the problems, not from reading about it beforehand.
- **"What would you do differently?"** → Add persistent storage with something like RocksDB so data survives restarts, and improve configuration to avoid hardcoded IPs and ports.
- **"Explain chain replication simply."** → Writes go Head → Mid → Tail, only acknowledged after reaching Tail. Reads come from Tail only, so you always get the latest committed state.
- **"Why C++?"** → Course requirement for this project.

---

8. PAST EXPERIENCE WALKTHROUGH

Version A — Quick (~30 seconds), use if asked casually:

At IDX India, I was a software engineer on the product-support team for about a year. Our company had an internal platform built on top of Drupal, and we used it to build and maintain investor-relations websites for publicly listed companies. My team's job was to keep the product healthy — resolving issues, handling version upgrades, configuring backends, and making sure client websites stayed up and running.

Version B — Detailed, only if they ask to go deeper:

I worked at IDX India for about a year as a software engineer. The company builds investor-relations websites for publicly listed companies — so think financial disclosures, shareholder communications, that kind of thing. I was on the product-support team. Our company had its own platform built on Drupal, and my team was responsible for keeping it running smoothly across 20+ client websites.

Day to day, I handled a mix of things. On the support side, I managed SSO access, took JIRA tickets from developers and client managers, helped troubleshoot platform issues, and was responsible for the post-upgrade QA checklist — I had to green-light sites before the hosting team would map them to production. I also got some exposure to React and Vue.js — there was a team converting an internal tool from .NET to React and Vue, and I worked alongside them and contributed where I could.

The experience that shaped me most was learning how to communicate well across teams and how to be disciplined about process — especially when people wanted shortcuts.

---

9. PROJECT OVERVIEWS

Distributed File Storage System (C++): We built a low-level distributed file storage system with fault tolerance. It takes a file, breaks it into chunks, and stores them on different storage nodes. At retrieval time, it collects chunks from nodes, reconstructs the file, verifies integrity by comparing hashes, and returns the final file. *Why?* I wanted to understand how distributed systems actually work under the hood. Theory tells you what to do, but building it teaches you why it's hard.

Calendar Event Manager (Java): It's a calendar event management system built in Java. The main focus was the design — clean MVC architecture, SOLID principles, and design patterns like Factory, Adapter, and Command. The goal was to make it easily extensible — if you want to add a new feature, you can do it without touching existing tested code. We also put heavy emphasis on testing — 98% mutation test coverage with 200+ JUnit tests.

Kambaz — Learning Management System (React, Node.js, MongoDB): Kambaz is a full-stack replica of Canvas LMS built as part of my web development coursework. It supports course management, assignments, grading, and real-time updates using WebSockets. It's the project where I got comfortable with the full stack — React frontend, Node/Express backend, MongoDB, REST APIs, and WebSockets all tied together.

---

10. QUESTIONS FOR THE MANAGER

1. "What does a typical day or week look like for an EDG intern?"
2. "What's one thing you wish you knew when you started at MathWorks?"
3. "Is there anything you'd recommend I learn or prepare before the internship starts?"

---

11. BACK-POCKET STORIES

Strengths / Process Discipline (SSO Story): At IDX, I managed SSO access for client environments. People would regularly come to me saying "I already spoke to your manager, just give me access." I always respectfully declined and asked them to raise a JIRA ticket first. It wasn't about being difficult — it was about accountability and having a paper trail.

Pressure / Deadlines: Two all-nighters for assignment deadline. *(Use brief version only if asked — keep it to 2-3 sentences.)*

Pivot Story: Changing direction mid-project. *(Use if asked about adaptability.)*

---

12. SECRET WEAPON — Use at the very end

When: After your questions are answered, say: "*Thank you for answering those. Before we wrap up, I'd love to share one quick thing that's personally meaningful to me about this opportunity...*"

In 1994, my dad was completing his master's thesis in mathematics on linear control theory. His professor was an ISRO scientist who visited the US for a conference and brought back a copy of MATLAB on a floppy disk. He passed it to my dad, and my dad became the first student at his university to actually use MATLAB — most students only knew about it theoretically. He completed his thesis on it.

When I called my dad to tell him I had an interview at MathWorks — the company that makes MATLAB — he got goosebumps. He told me this story and we both had chills. So landing this internship wouldn't just be a professional achievement for me — it would be a personal one for my whole family.

Keep this under 30 seconds. Tell it, let it land, say thank you, and end.

---

13. QUICK REFERENCE — QUESTION → STORY MAP

Manager asks...	Use story...
"Tell me about yourself"	#1 Introduction
"Why MathWorks / EDG?"	#2 Why MathWorks
"Time you took initiative"	#3 Automation (IDX)
"Something you're proud of"	#3 Automation (IDX)

Manager asks...	Use story...
"Leadership"	#3 Automation (IDX)
"Conflict with a teammate"	#4 Testing disagreement (Calendar)
"A time you failed / messed up"	#5 Client escalation (IDX)
"Negative feedback"	#5 Client escalation or #6 Not asking for help
"What's your weakness?"	#6 Not asking for help (IDX)
"How do you develop yourself?"	#6 Not asking for help (IDX)
"Learned a new skill"	#7 Distributed File System
"Outside your comfort zone"	#7 Distributed File System
"Tell me about past experience"	#8 IDX walkthrough
"Walk me through a project"	#9 Project overviews
"Your strengths?"	#11 SSO process discipline
"Handling pressure/deadlines"	#11 All-nighters
"Anything else / final thoughts?"	#12 Dad's MATLAB story

---

14. REMINDERS

- **Tone will be conversational** — don't let that make you casual
- **Expect follow-ups on everything** — the manager will dig in
- **Be honest** when you don't know something — say so confidently
- **Use "I" not "we"** when describing your specific contributions
- **Keep answers to 1-2 minutes** — don't monologue
- **The interview is ~25-30 minutes** — pace yourself
- **Manager may reference your HireVue recording** — stay consistent

You've got this, Mahip. Go get it. 🎯